SLOWKING — Research Roadmap

Search over Learned Opponent-belief World, Knowledge-Intensive Nash Game-solver

Version 0.1 · 2026-07-23 · Will Hooper · ABRA Tier-3

> *These are not decorations. Each tag in Slowking's Library is a **paper I will write and a component I will build** on the road to a real Tier-3 solver. This document turns the five research pillars into an ordered plan: what each paper proves, what it produces in code, and how it plugs into the pipeline. SLOWKING is the hard one — a genuine research effort — and this is how it gets built, honestly, one pillar at a time. The math foundation is already laid in the simulator white paper; this is the execution plan on top of it.*

The target

Champions is a two-player, zero-sum, **simultaneous-move**, **imperfect-information** stochastic game. Solving it well means: track what the opponent is hiding (a belief), look ahead over how the game could branch, and play the move that is best against their best response — a **mixed-strategy equilibrium**, not a single "best move." SLOWKING is the model that does that. No single paper gets us there; five do, in order.

The five papers (each a real deliverable)

Paper 1 — POSG / Markov game: the formalism and the offline dataset

Proves: that a Champions battle is a partially-observable stochastic game, and that our logged replays are a valid offline dataset of trajectories `(s, o, a, r)` to learn from. **Produces:** `engine/game-spec.js` — a precise state/observation/action encoding, and a converter that turns every stored replay (we already keep the per-turn stream + raw logs) into training trajectories. **Status:** the data exists (30k+ turn transitions); the encoding is the first build.

Paper 2 — Learned dynamics / world model: `Tθ(s' | s, a)`

Proves: we can learn the transition kernel of the closed Champions engine from logged transitions alone, grey-boxed with CHOMP's exact damage as a prior so the model only has to learn what CHOMP doesn't (status chains, secondary procs, switch dynamics, position). **Produces:** a trained forward model that, given a state and a joint action, predicts the next state distribution — the thing we cannot query from the real engine. Calibrated against the observed damage distributions (`data/dynamics.json`) and observed move order. **Literature:** MuZero (learned model + planning), Dreamer (latent world models), Metamon (reconstructing 5M+ Showdown trajectories offline).

Paper 3 — Mixed Nash under simultaneous moves: the per-turn subgame solver

Proves: each turn is a matrix game whose solution is a mixed equilibrium; we can approximate it tractably at VGC branching factors. **Produces:** a subgame solver (regret-matching / CFR-style) that outputs a *distribution* over actions, so SLOWKING plays unexploitably instead of deterministically — the

thing that makes it robust to a thinking opponent. **Literature:** CFR (Zinkevich et al.), and its modern deep variants.

Paper 4 — Belief-state search: planning over what the opponent hides

Proves: searching over belief states (distributions consistent with what's been revealed), not raw states, is both necessary (imperfect info) and feasible with the learned model. **Produces:** the search itself — a few plies deep, maintaining and updating the opponent belief from each observation, combining Papers 2 and 3. This is the core of `engine/slowking.py`'s `belief_search`, currently a stub. **Literature:** ReBeL, Player-of-Games, Student-of-Games (belief-state search that reaches superhuman play in imperfect-information games).

Paper 5 — Offline RL value/policy: making it strong without a live engine

Proves: we can train a value head and policy from logged data without ever interacting with the real engine, avoiding the distributional-shift traps that sink naive offline RL. **Produces:** the value function `V(belief)` and a policy prior that guides the search (so we prune to the few viable lines instead of simulating every path) — the piece that makes SLOWKING fast enough to be useful. **Literature:** CQL, IQL, Decision Transformer; AlphaStar / Gumbel-MuZero for policy-guided pruning.

How they compose

```
Paper 1 (spec + data)  ->  Paper 2 (world model Tθ)  ->  Paper 4 (belief search)
                                    ^                           ^
                       Paper 5 (value/policy) ----------------/
                                   Paper 3 (mixed-Nash subgame solver) feeds the search's per-turn cho
```

SLOWKING = Paper 2's model + Paper 5's value/policy, searched by Paper 4 over beliefs, with Paper 3 solving each turn's subgame. Papers 1 and 5 are the near-term, data-ready builds; 2–4 are the research.

How SLOWKING pays off the moment it exists

- **KADABRA** gets the deep line — "the equilibrium-best play here was X" — with no interface change.
- **DITTO** gains a Tier-3 finalist vetter: JOLTEON screens thousands, MEDICHAM re-ranks finalists, SLOWKING vets the last few. (Coarse-to-fine — being wired now with MEDICHAM as the middle tier.)
- **MEDICHAM** rollouts get a smarter default policy (Paper 5's policy prior) than the behaviour-clone.

Honest status

Zero of the five are trained today; the interface (`engine/slowking.py`) and the data pipeline that feeds them are built and the corpus is growing hourly. Papers 1 and 5 are startable now on current data. This is a multi-month research track, stated plainly — not a claim that it is done.

Related. Simulator white paper · Cheat sheet · Executive summary